INTRODUCTION

Card Games are a traditional use of the principles of computer science, where data is processed and simulated efficiently and randomly. Blackjack, otherwise known as Twenty-One, is one of the most popular casino banking games in the world. Unlike other card games, the approach here is such that Blackjack is a comparing card game played by a player and a dealer.

Blackjack, a betting game involving a casino banker, will be the basis of this case study. It will illustrate how a simulation of a Deck of Cards can be done through a Linear Data Structure; that is, through a Stack. A Player plays against a Dealer, who tries to get cards that sum to 21 or the closest to 21 without going "bust." This report will outline the concept and programming of this game through the C programming language.

PROBLEM DESCRIPTION

In a normal game of Blackjack, the deck of 52 playing cards needs to be randomly shuffled and disposed of in a Last In-First Out pattern. The conventional management of an array may not be the most efficient process when it comes to the distribution of the playing cards.This is the reason, the Stack datastructure is used in this implementation.

The Problem: To handle card operations on a deck as well as game operations efficiently without any memory leak or bias.

The Context: The dealer (computer) has strict rules (hit if 17), while the player has interactives. It requires a good mechanism for handling the 'Ace' card value dynamically, either 1 or 11.

Game Logic Complexity: Handling the dynamic valuation of specific cards (e.g., the Ace, which can be valued at 1 or 11) to prevent the player from "busting" unnecessarily, while simultaneously enforcing strict dealer rules (e.g., hitting on soft 16).

OBJECTIVES OF THE CASE STUDY

1. **To implement a Stack data structure** for managing the deck of cards, utilizing PUSH (add card) and POP (deal card) operations.
2. **To apply the Fisher-Yates Shuffle algorithm** to ensure an unbiased, random permutation of the deck.
3. **To simulate real-world game logic**, including specific Blackjack rules for dealers and players.
4. **To demonstrate memory management** in C through the use of pointers and structural definitions.

PROPOSED SOLUTION [ALGORITHM]

Step 1: Start Begin the program execution.

Step 2: Define Data Structures

- Define a structure Card with properties: suit, name, and value.
- Create a global array deck of size 52 to act as the stack.
- Initialize an integer top to -1 to serve as the stack pointer.

Step 3: Initialize Deck

- Define arrays for card suits ("Hearts", "Diamonds", etc.), names ("2", "3", ... "Ace"), and values.
- Iterate through all 4 suits and 13 ranks.
- For each combination, increment top and store the new Card in deck[top] (Push operation).

Step 4: Shuffle Deck

- Seed the random number generator using the current time.
- Iterate i from the top of the deck down to 1.
- Generate a random index j between 0 and i.
- Swap the card at deck[i] with the card at deck[j].

Step 5: Initial Deal

- Pop the top card from deck and add to playerHand.
- Pop the top card from deck and add to dealerHand.
- Pop the top card from deck and add to playerHand (2nd card).
- Pop the top card from deck and add to dealerHand (2nd card).

Step 6: Player Turn Loop

- **Step 6.1:** Calculate Player Score. Sum card values; if total > 21 and Aces exist, subtract 10 for each Ace until total <= 21.
- **Step 6.2:** Display Dealer's first card (visible card) and Player's full hand/score.
- **Step 6.3:** Check Score Conditions:
  - If Score == 21: Display "Blackjack" and Go to Step 8.
  - If Score > 21: Display "Bust" and Go to Step 9 (End Game).
- **Step 6.4:** Read User Input ('h' for Hit, 's' for Stand).
  - If 'h': Pop card from deck, add to playerHand, Repeat Step 6.
  - If 's': Go to Step 7.

Step 7: Dealer Turn Loop

- **Step 7.1:** Calculate Dealer Score.
- **Step 7.2:** Display Dealer's full hand and score.
- **Step 7.3:** Check Dealer Rules:
    - If Score >= 17: Dealer Stands. Go to Step 8.
    - If Score < 17: Dealer Hits. Pop card from deck, add to dealerHand, Repeat Step 7.

Step 8: Determine Result

- Compare final Player Score (pScore) vs Dealer Score (dScore).
    - If dScore > 21: Display "Dealer Busted! YOU WIN!".
    - Else If pScore > dScore: Display "YOU WIN!".
    - Else If dScore > pScore: Display "DEALER WINS".
    - Else: Display "It's a Tie".

Step 9: Stop Terminate the program.

IMPLEMENTATION [CODE]

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#define MAX_CARDS 52

typedef struct {
    char *suit;
    char *name;
    int value;
} Card;
Card deck[MAX_CARDS];
int top = -1;

void push(char *suit, char *name, int value) {
    if(top < MAX_CARDS - 1) {
        top++;
        deck[top].suit = suit;
        deck[top].name = name;
        deck[top].value = value;
    }
}

Card pop() {
     if (top == -1) {
         printf("Error : Disk Empty\n");
         exit(1);
      }
    return deck[top--];
}
```

```
// Fisher-Yates Shuffle Algorithm
void shuffleDeck() {
    srand(time(0));
    for (int i = top; i > 0; i--) {
        int j = rand() % (i + 1);
        Card temp = deck[i];
        deck[i] = deck[j];
        deck[j] = temp;
    }
}
void initDeck() {
    char *suits[] = {"Hearts", "Diamonds", "Clubs", "Spades"};
    char *names[] = {"2", "3", "4", "5", "6", "7", "8", "9", "10", "Jack", "Queen", "King", "Ace"};
    int values[] = {2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10, 11};

    top = -1; // Reset stack pointer
    for (int s = 0; s < 4; s++) {
        for (int n = 0; n < 13; n++) {
            push(suits[s], names[n], values[n]);
        }
    }
}
int calculateScore(Card hand[], int count) {
    int score = 0;
    int aces = 0;
    for(int i=0; i<count; i++) {
        score += hand[i].value;
        if(hand[i].value == 11) aces++;
    }

    // If bust, convert Aces from 11 to 1
    while(score > 21 && aces > 0) {
```

```
        score -= 10;
        aces--;
    }
    return score;
}

// --- MAIN GAME LOOP ---
int main() {
    Card playerHand[10], dealerHand[10];
    int pCount = 0, dCount = 0;
    char choice;

    // 1. Setup Game
    initDeck();
    shuffleDeck();

    printf("\n==========================================\n");
    printf("     BLACKJACK (Stack Implementation)     \n");
    printf("==========================================\n");

    // 2. Initial Deal (Pop 2 cards each)
    playerHand[pCount++] = pop();
    dealerHand[dCount++] = pop();
    playerHand[pCount++] = pop();
    dealerHand[dCount++] = pop();

    // 3. Player Turn
    while(1) {
        int score = calculateScore(playerHand, pCount);

        printf("\n--- YOUR TURN ---\n");
        printf("Dealer shows: %s of %s\n", dealerHand[0].name, dealerHand[0].suit);
```

```
    printf("Your Hand:    ");
    for(int i=0; i<pCount; i++) printf("[%s %s] ", playerHand[i].name, playerHand[i].suit);
    printf("\nYour Score:   %d\n", score);
    if(score == 21) {
        printf("\n*** BLACKJACK! You got 21! ***\n");
        break;
    }
    if(score > 21) {
        printf("\n*** BUST! You went over 21. YOU LOSE. ***\n");
        return 0; // End game immediately
    }
    printf("Action: (h) Hit or (s) Stand? ");
    scanf(" %c", &choice);
    if(choice == 'h' || choice == 'H') {
        printf("Dealing card...\n");
        playerHand[pCount++] = pop(); // Pop from stack
    } else if(choice == 's' || choice == 'S') {
        printf("You chose to Stand.\n");
        break;
    } else {
        printf("Invalid input.\n");
    }
}
// 4. Dealer Turn
printf("\n==========================================\n");
printf("        DEALER'S TURN   (AI)        \n");
printf("==========================================\n");

// Reveal hidden card
printf("Dealer reveals: %s of %s\n", dealerHand[1].name, dealerHand[1].suit);

while(1) {
```

```
        int dScore = calculateScore(dealerHand, dCount);
        printf("Dealer Hand: ");
        for(int i=0; i<dCount; i++) printf("[%s %s] ", dealerHand[i].name, dealerHand[i].suit);
        printf(" -> Score: %d\n", dScore);

        if(dScore >= 17) {
            printf("Dealer Stands (Score is 17 or higher).\n");
            break;
        } else {
            printf("Dealer Hits (Score < 17)...\n");
            dealerHand[dCount++] = pop(); // Pop from stack
        }
    }
    int pScore = calculateScore(playerHand, pCount);
    int dScore = calculateScore(dealerHand, dCount);
    printf("\n==========================================\n");
    printf("FINAL: You [%d] vs Dealer [%d]\n", pScore, dScore);
    if(dScore > 21) {
        printf("RESULT: Dealer Busted! YOU WIN!\n");
    } else if(pScore > dScore) {
        printf("RESULT: You have the higher score. YOU WIN!\n");
    } else if(dScore > pScore) {
        printf("RESULT: Dealer has the higher score. DEALER WINS.\n");
    } else {
        printf("RESULT: It's a Tie (Push).\n");
    }
    printf("==========================================\n");
    return 0;
}
```

OUTPUT AND RESULTS

```
PS C:\Users\Arun\Desktop\Blackjack> .\black-jack.exe

========================================
     BLACKJACK (Stack Implementation)
========================================

--- YOUR TURN ---
Dealer shows: [2 of Clubs] (Visible Score: 2)
Your Hand:    [5 Clubs] [Ace Diamonds]
Your Score:   16
Action: (h) Hit or (s) Stand? s

========================================
          DEALER'S TURN (AI)
========================================
Dealer Hand: [2 Clubs] [10 Clubs]  -> Score: 12
Dealer Hits...
Dealer Hand: [2 Clubs] [10 Clubs] [9 Clubs]  -> Score: 21

========================================
FINAL: You [16] vs Dealer [21]
RESULT: DEALER WINS.
========================================
```

```
PS C:\Users\Arun\Desktop\Blackjack> .\black-jack.exe

========================================
     BLACKJACK (Stack Implementation)
========================================

--- YOUR TURN ---
Dealer shows: [Queen of Spades] (Visible Score: 10)
Your Hand:    [10 Diamonds] [6 Spades]
Your Score:   16
Action: (h) Hit or (s) Stand? s

========================================
          DEALER'S TURN (AI)
========================================
Dealer Hand: [Queen Spades] [Queen Clubs]  -> Score: 20

========================================
FINAL: You [16] vs Dealer [20]
RESULT: DEALER WINS.
========================================
```

```
PS C:\Users\Arun\Desktop\Blackjack> .\black-jack.exe

========================================
     BLACKJACK (Stack Implementation)
========================================

--- YOUR TURN ---
Dealer shows: [3 of Hearts] (Visible Score: 3)
Your Hand:    [10 Diamonds] [10 Hearts]
Your Score:   20
Action: (h) Hit or (s) Stand? s

========================================
          DEALER'S TURN (AI)
========================================
Dealer Hand: [3 Hearts] [10 Spades]  -> Score: 13
Dealer Hits...
Dealer Hand: [3 Hearts] [10 Spades] [King Clubs]  -> Score: 23

========================================
FINAL: You [20] vs Dealer [23]
RESULT: YOU WIN!
========================================
PS C:\Users\Arun\Desktop\Blackjack>
```

```
PS C:\Users\Arun\Desktop\Blackjack> .\black-jack.exe

========================================
     BLACKJACK (Stack Implementation)
========================================

--- YOUR TURN ---
Dealer shows: [Queen of Spades] (Visible Score: 10)
Your Hand:    [Jack Hearts] [10 Clubs]
Your Score:   20
Action: (h) Hit or (s) Stand? s

========================================
          DEALER'S TURN (AI)
========================================
Dealer Hand: [Queen Spades] [7 Hearts]  -> Score: 17

========================================
FINAL: You [20] vs Dealer [17]
RESULT: YOU WIN!
========================================
```

ANALYSIS AND DISCUSSION

Time Complexity and Space Complexity Analysis

- **Initialization:** O(1) (Specifically O(52), which is constant).
- **Shuffle:** O(N) where N is 52. The loop runs exactly 51 times, performing one constant-time swap per iteration.
- **Dealing (Pop):** O(1). Decrementing an integer and returning an array index is an immediate CPU operation.
- **Total Space Complexity: O(N)**, where N is the number of cards.

Overall System Performance: The game is extremely lightweight and runs instantly on any modern hardware.

FUTURE SCOPE

1. Implementation of a Betting System : Currently, the game tracks wins and losses but does not involve currency. A future iteration could introduce a Virtual Economy

2. Advanced Game Rules (Splitting and Doubling Down)

3. Multiplayer: Using Socket Programming in C to allow two players on different computers to play against the same dealer over a local network.

CONCLUSION

A solid example comes from the Blackjack game setup showing how basic data layouts work in live simulations. Using C, the effort connected classroom ideas about computing to hands-on software building.

A single idea drove everything: stacking cards like real decks made perfect sense. Not only did it mirror how players handle hands, but also sped up draws dramatically. Instead of shuffling through lists each time, grabbing the top card took constant time. What stood out was how naturally theory fit practice - no forced logic, just clean matching of behavior.

Putting it all together, this work shows clearly how basic data layouts help build software that runs well and handles tasks reliably. What matters most is picking the correct layout - like Stack instead of Array - depending on what the task needs, such as popping items from the top. Working through these steps brings out key abilities: handling memory carefully, building logical routines, organizing code in clear ways. These are the pieces everything else stands on when moving into tougher areas like creating complex programs or games.